

Concurrency Interview Questions

Thread, Mutex, Race Condition, Deadlock, Atomic — ամբողջական ժողովածու

Այս ֆայլը concurrency-ի interview հարցերի ժողովածու է code օրինակներով (սխալ + ճիշտ լուծում): Երեք մաս.

Մաս A — 30 multiple-choice: Մաս B — 25 բաց (open) հարց: Մաս C — 15 code / «ի՞նչն է խնդիրը, ինչպես լուծել»:

Բոլոր պատասխանները **միայն վերջում** են (Answer Key A, B, C):

ՄԱՍ A — Multiple Choice (30)

Q1

Thread ի՞նչ է: - (a) առանձին ծրագիր - (b) ծրագրի ամենափոքր կատարողական միավոր - (c) memory block - (d) file

Q2

Process vs thread - memory: - (a) process shared, thread առանձին - (b) process առանձին, thread shared - (c) երկուսն էլ shared - (d) երկուսն էլ առանձին

Q3

Որ մեկն է ավելի թեթև ստեղծել: - (a) process - (b) thread - (c) նույնը - (d) ոչ մեկը

Q4

std::thread-ի join() ի՞նչ է անում: - (a) thread ջնջում - (b) սպասում մինչև thread ավարտվի - (c) նոր thread - (d) detach

Q5

detach() ի՞նչ է: - (a) սպասում - (b) thread դառնում անկախ (background) - (c) ջնջում - (d) lock

Q6

thread object ոչնչանալուց առաջ չկա join/detach -> ? - (a) OK - (b) std::terminate - (c) leak - (d) deadlock

Q7

Mutex ի՞նչ է: - (a) memory - (b) Mutual Exclusion - մեկ thread միանգամից critical section - (c) thread - (d) pointer

Q8

Mutex ի՞նչ խնդիր է լուծում: - (a) deadlock - (b) race condition - (c) leak - (d) crash

Q9

lock_guard-ի առավելությունը: - (a) արագ - (b) RAII - lock ctor/unlock dtor (exception-safe) - (c) քիչ memory - (d) atomic

Q10

lock_guard vs unique_lock: - (a) նույնը - (b) unique_lock ճկուն (ձեռքով lock/unlock, condition_variable) - (c) lock_guard ճկուն - (d) unique_lock հին

Q11

Race condition ի՞նչ է: - (a) thread արագ - (b) մի քանի thread նույն data փոխում, արդյունք կարգից կախված - (c) deadlock - (d) leak

Q12

counter++ ինչո՞ւ է race: - (a) atomic է - (b) 3 քայլ (read/+1/write) - thread-ներ խառնվում - (c) դանդաղ - (d) չկա race

Q13

Race condition-ի լուծում: - (a) mutex / atomic - (b) ավելի thread - (c) detach - (d) ոչինչ

Q14

Data race ի՞նչ է: - (a) լոգիկական bug - (b) sync-ից առանց նույն memory (write) -> UB - (c) deadlock - (d) leak

Q15

Race condition հայտնաբերող գործիք: - (a) valgrind memcheck - (b) ThreadSanitizer - (c) gprof - (d) gdb

Q16

Deadlock ինչ է: - (a) thread crash - (b) երկու+ thread սպասում իրար անվերջորեն - (c) leak - (d) race

Q17

Deadlock-ի պատճառ: - (a) նույն կարգով lock - (b) տարբեր կարգով մի քանի lock (circular wait) - (c) atomic - (d) detach

Q18

Coffman conditions-ի քանակը: - (a) 2 - (b) 3 - (c) 4 - (d) 5

Q19

Deadlock-ի ամենապարզ լուծում: - (a) ավելի mutex - (b) միշտ նույն կարգով lock - (c) detach - (d) atomic

Q20

std::lock ինչ է անում: - (a) մեկ mutex - (b) մի քանի mutex atomic (deadlock-free) - (c) unlock - (d) atomic

Q21

scoped_lock (C++17): - (a) մեկ mutex RAI - (b) մի քանի mutex RAI, deadlock-free - (c) atomic - (d) thread

Q22

atomic ինչ է: - (a) mutex - (b) անբաժանելի operation, race առանց mutex - (c) thread - (d) lock

Q23

atomic vs mutex - երբ atomic: - (a) բարդ logic - (b) փոքր operation (counter/flag) - (c) container փոխում - (d) միշտ mutex

Q24

atomic-ի առավելությունը mutex-ի նկատմամբ: - (a) բարդ logic - (b) ավելի արագ (lock-free, չկա blocking) - (c) քիչ memory - (d) sorted

Q25

compare_exchange (CAS) ինչ է: - (a) atomic add - (b) եթե == expected -> set desired (lock-free հիմք) - (c) mutex - (d) load

Q26

recursive_mutex ինչ է: - (a) սովորական - (b) նույն thread մի քանի lock - (c) atomic - (d) timed

Q27

Որ մեկն է blocking: - (a) atomic - (b) mutex lock (սպասում) - (c) load - (d) store

Q28

atomic flag ինչի համար: - (a) counter - (b) stop/ready signal thread-ների միջև - (c) mutex - (d) lock

Q29

Critical section ինչ է: - (a) main - (b) կող, որ միայն մեկ thread միանգամից պետք է մտնի - (c) thread - (d) mutex

Q30

Thread-ի հիմնական առավելությունը: - (a) քիչ memory - (b) parallelism (multi-core) + responsiveness - (c) sorted - (d) safety

ՄԱՍ B — Open Questions (25)

1. Thread ինչ է, process vs thread:
2. Thread ինչպես ես ստեղծում (std::thread), join/detach:
3. Ինչո՞ւ պետք է join/detach thread ոչնչանալուց առաջ:
4. Mutex ինչ է, ինչ խնդիր է լուծում:
5. lock_guard vs unique_lock:
6. Mutex-ի տեսակները (recursive, timed, shared):

7. Race condition թնչ է, ինչու է լինում:
8. Ինչո՞ւ counter++ race է (ներհատուկ):
9. Race condition ինչպես ես լուծում:
10. Data race vs race condition:
11. Race condition ինչպես ես հայտնաբերում:
12. Deadlock թնչ է, ինչու է լինում:
13. Coffman 4 conditions:
14. Deadlock ինչպես ես խուսափում (նույն կարգ, std::lock):
15. std::lock / scoped_lock:
16. Atomic թնչ է, ինչպես է լուծում race-ը:
17. Atomic vs mutex - երբ որը:
18. compare_exchange (CAS):
19. Memory order (հիմնունքներ):
20. atomic flag-ի օգտագործում:
21. Critical section թնչ է:
22. Ինչո՞ւ multithreading դժվար է (debugging):
23. Thread-safe թնչ է նշանակում:
24. Immutable data ինչո՞ւ է helpful concurrency-ում:
25. Thread-local storage թնչ է:

ՄԱՍ C — Code: «Թնչն է խնդիրը, ինչպես լուծել» (15)

C1

```
int counter = 0;
void work() { for(int i=0;i<1000000;i++) counter++; }
std::thread t1(work), t2(work);
```

Ի՞նչն է խնդիրը: - (a) ոչ մի - (b) race condition (counter shared, ոչ atomic) - (c) deadlock - (d) leak

C2

```
std::thread t(work);
// join/detach չկա
```

- (a) OK

- (b) std::terminate thread ոչնչանալուց
- (c) leak
- (d) race

C3

```
std::mutex m1, m2;
void A() { lock_guard l1(m1); lock_guard l2(m2); }
void B() { lock_guard l1(m2); lock_guard l2(m1); }
```

- (a) OK
- (b) deadlock (տարբեր կարգ)
- (c) race
- (d) leak

C4 (C3-ի լուծում)

Ինչպես լուծել C3-ը: - (a) ավելի mutex - (b) նույն կարգով lock (երկուսն էլ m1->m2) կամ std::scoped_lock(m1,m2) - (c) detach - (d) atomic

C5

```
std::mutex m;
void f() {
    m.lock();
    if (error) return;    // <-- ?
    m.unlock();
}
```

- (a) OK
- (b) error-ի դեպքում unlock չի հասնի -> mutex զբաղված (կամ deadlock)
- (c) race
- (d) leak

C6 (C5-ի լուծում)

Ինչպես լուծել C5-ը: - (a) ձեռքով unlock ամեն path-ում - (b) std::lock_guard (RAII, ավտոմատ unlock) - (c) atomic - (d) detach

C7

```
std::atomic<int> counter{0};  
void work() { for(int i=0;i<100000;i++) counter++; }
```

- (a) race condition
- (b) OK (atomic, չկա race)
- (c) deadlock
- (d) leak

C8

```
bool stop = false;           // սովորական bool  
void worker() { while(!stop) doWork(); }  
void ctrl() { stop = true; }
```

- (a) OK
- (b) data race (stop shared, ոչ atomic/synchronized)
- (c) deadlock
- (d) leak

C9 (C8-ի լուծում)

- (a) mutex ամեն կարողալուց
- (b) std::atomic stop
- (c) detach
- (d) ոչինչ

C10

```
std::vector<int> v;          // shared  
void add() { v.push_back(1); } // 2 thread
```

- (a) OK
- (b) race condition (vector ոչ thread-safe) -> mutex պետք
- (c) deadlock
- (d) atomic բավական

C11

```
std::recursive_mutex m;  
void f() { lock_guard<recursive_mutex> l(m); f(); }
```

recursive_mutex-ը ինչո՞ւ: - (a) սովորական mutex -> deadlock recursion-ի ժամանակ - (b) արագ - (c) atomic - (d) leak

C12

```
std::thread t([](){ cout << "hi"; });  
t.detach();  
// main ավարտվում
```

- (a) միշտ "hi" տպվում
- (b) կարող է չտպվել (main ավարտվում, detached thread չի հասնի)
- (c) CE
- (d) deadlock

C13

```
std::atomic<int> a{5};  
int exp = 5;  
a.compare_exchange_strong(exp, 10);  
cout << a;
```

- (a) 5
- (b) 10 (a==exp -> set 10)
- (c) 0
- (d) CE

C14

```
std::mutex m;  
void f() { lock_guard l(m); longIO(); } // երկար I/O lock-ի մեջ
```

Ի՞նչն է խնդիրը: - (a) ոչ մի - (b) երկար lock -> այլ thread-ների սպասում (performance) -- lock-ը փոքրացրու - (c) deadlock - (d) race

C15

```
std::scoped_lock lock(m1, m2); // C++17
```

- (a) deadlock հնարավոր
- (b) deadlock-free մի քանի mutex lock (RAII)
- (c) CE
- (d) միայն մեկ mutex

ANSWER KEY A

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b · 11-b · 12-b · 13-a · 14-b · 15-b · 16-b · 17-b · 18-c · 19-b · 20-b · 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b

ANSWER KEY B (համառոտ)

1. Thread՝ ծրագրի ամենափոքր կատարողական միավոր. Process առանձին memory (ծանր, isolated); thread shared memory (թեթև, մեկ process):
2. std::thread t(func); t.join() սպասում; t.detach() անկախ. Argument-ներ՝ t(func, arg):
3. thread object ոչնչանալուց առաջ պետք է join/detach; հակառակ -> std::terminate:
4. Mutual Exclusion՝ մեկ thread միանգամից critical section. Լուծում՝ race condition:
5. lock_guard պարզ (չի բացվում, lock ctor/unlock dtor); unique_lock ճկուն (ձեռքով lock/unlock, condition_variable):
6. mutex (սովորական), recursive_mutex (նույն thread մի քանի lock), timed_mutex (try_lock_for), shared_mutex (read/write, C++17):
7. Մի քանի thread նույն shared data փոխում; արդյունք կախված thread-ների կարգից -> անկանխատեսելի:
8. counter++ 3 քայլ (read/+1/write); thread-ներ խառնվում -> lost update:
9. mutex (lock_guard) կամ atomic; immutable/thread-local data:
10. Data race՝ sync-ից առանց նույն memory write -> UB. Race condition՝ լոգիկական, կարգից կախված:
11. ThreadSanitizer (-fsanitize=thread), Helgrind, code review:

12. Երկու+ thread սպասում իրար անվերջորեն (կախված). Ինչու՝ տարբեր կարգով մի քանի lock -> circular wait:
13. Mutual exclusion, hold and wait, no preemption, circular wait (բոլոր 4 պետք են):
14. Նույն կարգով lock; std::lock/scoped_lock; try_lock timeout:
15. std::lock մի քանի mutex atomic (deadlock-free); scoped_lock (C++17) RAII + std::lock:
16. Անբաժանելի operation -> race առանց mutex, փոքր operation-ների համար:
17. atomic արագ (lock-free), փոքր operation (counter/flag); mutex բարդ critical section (blocking):
18. compare-and-swap՝ եթե արժեքը == expected, set desired atomic; lock-free հիմք:
19. Atomic operation-ների կարգավորման երաշխիք; default seq_cst (խիստ):
20. Stop/ready signal thread-ների միջև, առանց mutex:
21. Կող, որ միայն մեկ thread միանգամից պետք է մտնի (shared data փոխում):
22. Non-deterministic (կարգը անկանխատեսելի), reproduce դժվար, race/deadlock նուրբ:
23. Thread-safe՝ մի քանի thread-ից անվտանգ կանչվող (չկա race/corruption):
24. Immutable -> չկա փոխում -> չկա race (կարդալ միշտ անվտանգ):
25. Thread-local՝ ամեն thread իր առանձին copy-ն (thread_local keyword) -> չկա sharing -> չկա race:

ANSWER KEY C

C1-b · C2-b · C3-b · C4-b · C5-b · C6-b · C7-b · C8-b · C9-b · C10-b C11-a · C12-b · C13-b · C14-b · C15-b

Բացատրություններ (C)

C1 counter shared + ոչ atomic -> race condition. Լուծում՝ mutex/atomic: **C2** join/detach չկա -> std::terminate: **C3** տարբեր կարգով lock (A: m1->m2, B: m2->m1) -> deadlock: **C4** նույն կարգով lock կամ scoped_lock(m1,m2) -> deadlock-free: **C5** error return -> unlock չի հասնի -> mutex մնում զբաղված: **C6** lock_guard (RAII) -> ավտոմատ unlock ամեն path-ում: **C7** atomic counter -> չկա race -> OK: **C8** stop սովորական bool shared -> data race. Լուծում՝ atomic: **C9** std::atomic stop: **C10** vector ոչ thread-safe -> push_back race -> mutex պետք: **C11** recursion-ի ժամանակ նույն thread մի քանի lock -> սովորական mutex deadlock -> recursive_mutex: **C12** main ավարտվում -> detached thread կարող է չհասնի -> "hi" կարող է չտպվել: **C13** a==exp(5) -> set 10 -> a=10: **C14** երկար I/O lock-ի մեջ -> այլ thread-ներ երկար սպասում -> lock-ը փոքրացրու (I/O դուրս): **C15** scoped_lock մի քանի mutex deadlock-free RAI:

Ընդհանուր կանոններ

THREAD: join/detach ոչնչանալուց առաջ | shared memory -> race վտանգ

MUTEX: lock_guard (RAII) | նույն կարգով lock (deadlock խուսափել) | փոքր critical section

RACE: shared + ոչ atomic -> mutex/atomic | ThreadSanitizer

DEADLOCK: տարբեր կարգ -> circular wait | նույն կարգ / scoped_lock

ATOMIC: փոքր operation (counter/flag) արագ | բարդ -> mutex

vector/container ՈՉ thread-safe -> mutex պետք